# Week 3: Conditional Statements

**General Instructions:**

1. **Remove existing directories and create a fresh directory of your own. Write all the programs inside the newly created directory.**
2. **Remove the directory that you have created at the end of your Lab session.**
3. **Inform one of the TAs before you leave the Lab.**

---

**I**n the previous week, we have learnt the followings

1. What is a statement? We have seen two types of statement – simple statement/expression, and comments
2. Variable declaration
3. Different types of datatypes
4. The printf and scanf functions
5. The scope of a variable

In this week, we will learn another type of statement named **composite statement**. A composite statement can hold more than one simple/expression statement. There are different types of composite statements. This session will focus on two types of composite statement – **conditional statement and loop**.

## 1. Logical Operators/Conditional Operators

To understand conditional and loop statements, one needs to understand a kind of operator called **logical operator/Conditional Operators**. Unlike arithmetic operators that you have seen in the previous session, a logical operator forms a logical expression and returns a Boolean value – **True** or **False.**

**Binary Conditional Operators**
**Syntax: <operand> <operator> <operand>**

| Items | Operators | Examples | Return |
|---|---|---|---|
| Equal to | == | a == b; a == 10; 10 == 20 | If the value in the left side of the operator is equal to the value in the right side, it returns 1(true), else 0(false) |
| Less than | < | a < b; a < 10; 10 < 20 | If the value in the left side of the operator is less than the value in the right side, it returns 1(true), else 0(false) |
| Less than or equal to | <= | a <= b; a <= 10; 10 <= 20 | If the value in the left side of the operator is less than or equal to the value in the right side, it returns 1(true), else 0(false) |
| Greater than | > | a > b; a > 10; 10 > 20 | If the value in the left side of the operator is greater than the value in the right side, it returns 1(true), else 0(false) |
| Greater than or equal to | >= | a >= b; a >= 10; 10 >= 20 | If the value in the left side of the operator is greater than or equal to the value in the right side, it returns 1(true), else 0(false) |
| Not equal to | != | a != b; a != 10; 10 != 20 | If the value in the left side of the operator is not equal to the value in the right side, it returns 1(true), else 0(false) |

**Logical Operators**
**Syntax: <logical expression> <operator> <logical expression>**

| Items | Operators | Examples |
|---|---|---|
| AND | && | (a == 10) && (b< 20) |
| OR | \|\| | (a == 10) \|\| (b< 20) |
| NOT | ! | !(a < b) |

**Truth Tables of logical operators**

**AND (&&)**

| Input 1 | Input 2 | Output |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

**OR (||)**

| Input 1 | Input 2 | Output |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

**NOT (!)**

| Input | Output |
|---|---|
| 0 | 1 |
| 1 | 0 |

## 2. Conditional Statements

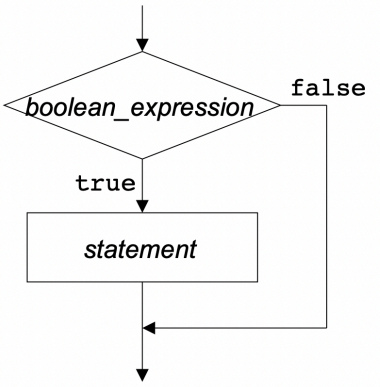
Conditional statements allow a program to make decisions based on the value of a variable or an expression. Commonly used conditional statements in C are - the if statement, the if-else statement, the if-else-if statement, and the switch statement. This session will focus on the if-else-if statement.

### 2.1. If statement

Syntax:

```
if (conditional statement){
 /* the statements to be executed if the conditional statement returns true *

}
```



**Examples**

a. Check for leap year

```
#include<stdio.h>

main(){
        int year;

        printf("Entre the year:");
        scanf("%d",&year);

        if((year % 4)==0){  // % operator represents modulo operator

        printf("The year %d is a Leap Year", year);

        }
}
```

In the above program, the if-statement checks if the value in the variable year is divisible by 4. If yes, it prints the message, otherwise not.

b. Find the largest number of three variables

```
#include <stdio.h>
int main(){
        int a, b, c;

        printf("Enter three numbers?");
        scanf("%d %d %d",&a,&b,&c);

        if(a>b && a>c) {
                printf("%d is largest",a);
        }
        if(b>a && b > c) {
                printf("%d is largest",b);
        }

        if(c>a && c>b) {
                printf("%d is largest",c); }

        if(a == b && a == c) {
                printf("All are equal");
        }
}
```

c. When you want to test if the value of a variable is in a range.

WRONG

```
int temp =10;
if (0 < temp < 100) {
        printf("Yes, it is in range");
}
```

The correct formation is

```
int temp =10;
if (0 < temp && temp< 100) {
        printf("Yes, it is in range");
}
```

d. If the if-body has only one statement, {…} bracket is not required. The following is also true.

```
        int temp =10;
        if (0 < temp && temp< 100)
                printf("Yes, it is in range");   // it is inside if-statement
```

e. When you want to test if the value of a variable is one of two alternates.

```
if (choice == 'M' || 'L') {   // this statement is wrong
        printf("You're correct!");
}


if (choice == 'M' || choice == 'L') { // This statement is correct
        printf("You're correct!");
}
```

f. An if-statement can be nested.

```
if (x > 0){
        if (y > 0) {
                z = 1;
        }
}
```

It can also be written as below.

```
if (x > 0)
        if (y > 0)
                z = 1;
```

Note: Proper indentation is important for proper visualization. Proper indentation helps in identifying several syntax or logical errors. You should always follow proper indentation.

### 2.2. The if-else statement

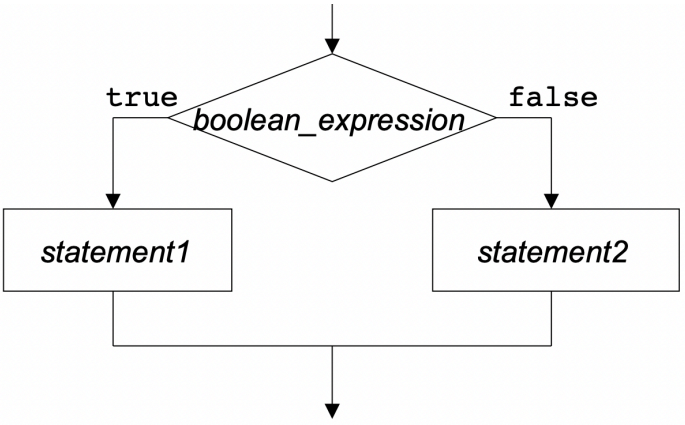
The if-else statement is used to perform two operations for a single condition. The if-else statement is an extension to the if statement using which, we can perform two different operations, i.e., one is for the correctness of that condition, and the other is for the incorrectness of the condition. Here, we must notice that if and else block cannot be executed simultaneously. Using if-else statement is always preferable, since it always invokes an otherwise case with every if condition.

Syntax:

```
if(expression) {

      //code to be executed if condition is true
}
else {

      //code to be executed if condition is false
}
```



Examples:

a. Find Leap Year

```
#include<stdio.h>

main(){
int year;

    printf("Entre the year:");
    scanf("%d",&year);

    if((year % 4)==0){  // % operator represents modulo operator

            printf("The year %d is a Leap Year", year);

    }
  else {
            printf("The year %d is NOT a Leap Year", year);
    }
}
```

b. Find if a number if odd or even

```
#include
int main() {
        int num;
        printf("Enter the number: ");
        scanf("%d",&num);
        if(num % 2) {
                printf("The number is odd");
      }
      else {
                printf("The number if even");
            }
}
```

c. **The Dangling else problem**

When an if statement is nested inside the then clause of another if statement, the else clause is paired with the closest if statement without an else clause.

```
if (x > 0)
        if (y > 0)
                z=0;
else    // note that this else goes with the second if-statement
        z=1;
```

It is always appropriate to properly indent the above program as below.

```
if (x > 0)
        if (y > 0)
                z=0;
        else    // note that this else goes with the second if-statement
                z=1;
```

If the else-statement is associated with the first if-statement, write as below

```
if (x > 0){
        if (y > 0)
                z=0;
}
else   // note that this else goes with the first if-statement
        z=1;
```

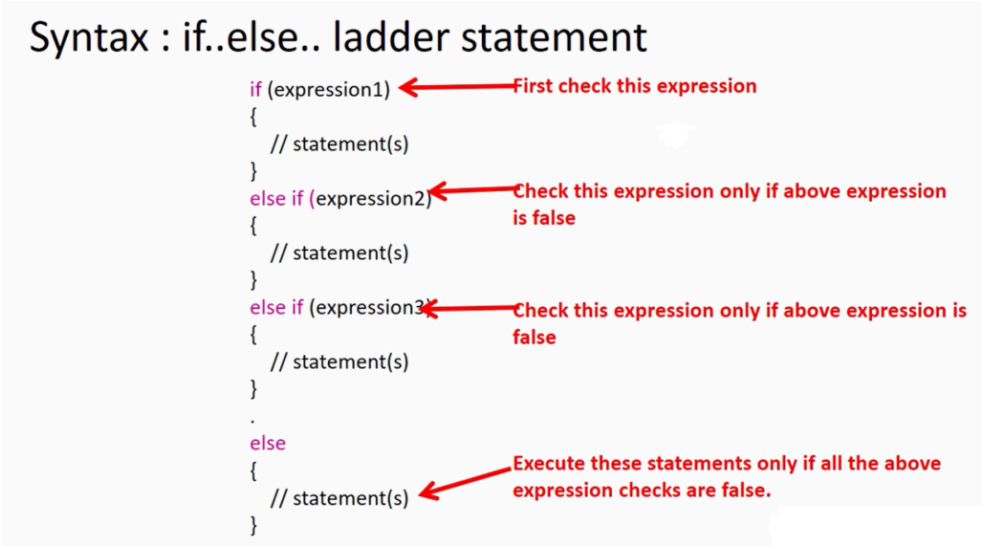
### 2.3. The if-else-if ladder statement

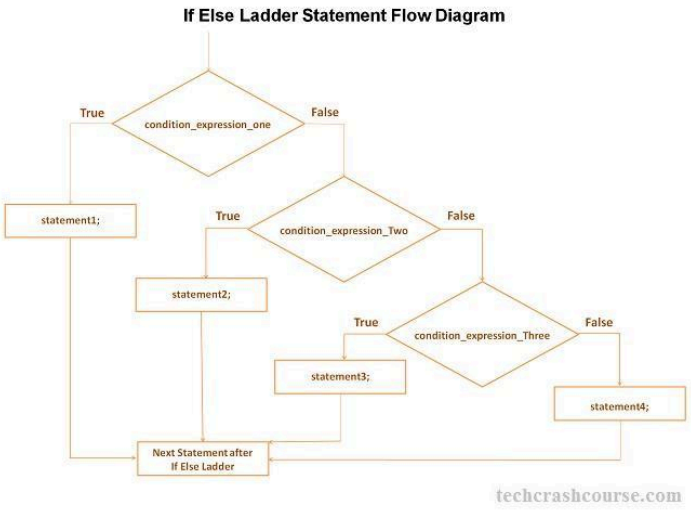
The if-else-if ladder statement is an extension to the if-else statement. It is used in the scenario where there are multiple cases to be performed for different conditions. In if-else-if statement, if a condition is true then the statements defined in the if block will be executed, otherwise if some other condition is true

then the statements defined in the else-if block will be executed, at the last if none of the condition is true then the statements defined in the else block will be executed. There are multiple else-if blocks possible. It is similar to the switch case statement where the default is executed instead of else block if none of the cases is matched.

**Syntax:**

```
if(condition1){
    //code to be executed if condition1 is true
}else if(condition2){
    //code to be executed if condition2 is true
} else if(condition3){
    //code to be executed if condition3 is true
}
    ...
else{
    //code to be executed if all the conditions are false
}
```



**Example:**

a. Find the largest number of three variables

```
#include <stdio.h>
int main(){
        int a, b, c;

        printf("Enter three numbers?");
        scanf("%d %d %d",&a,&b,&c);

        if(a>b){
         if (a > c){
                        printf("%d is largest",a);
                }
                else if(c > b) {
                        printf("%d is largest",c);

                }

        }
        else if(b > c) {
                        printf("%d is largest",b);
                }
                else if(c > a) {
                        printf("%d is largest",c);

                }
         else{
                printf("%d is largest",c);

        }
}
```

b.

3. Different indenting styles can be followed.

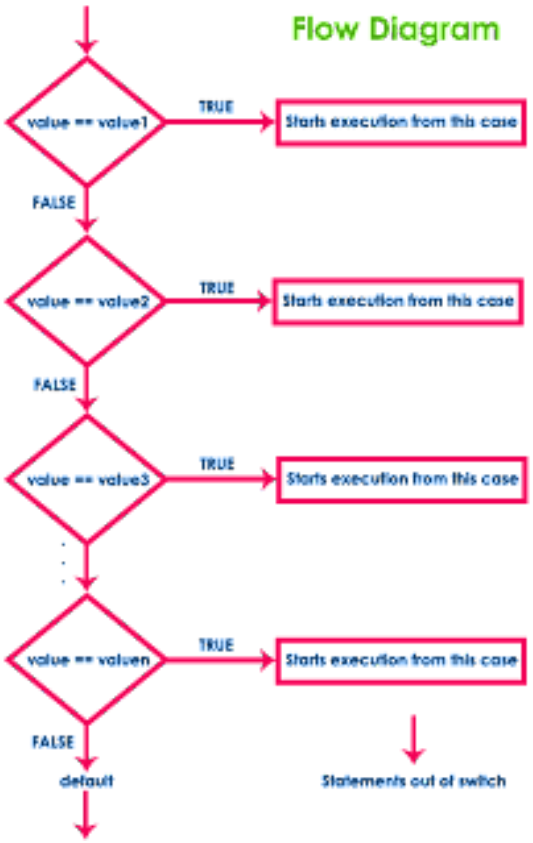
| | |
|---|---|
| • Single-line `if` statement: | `if (y > 0) color = "red";` |
| • Multi-line `if` statement: | `if (zipcode == 15213) {`<br>`  city = "Pittsburgh";`<br>`  state = "PA";`<br>`}` |
| • The `if-else` statement: | `if (temperature <= 32.0) {`<br>`  forecast = "SNOW";`<br>`}`<br>`else {`<br>`  forecast = "RAIN";`<br>`}` |
| • Multiple alternatives: | `if (value < 0) {`<br>`  valueType = "negative";`<br>`}`<br>`else if (value == 0) {`<br>`  valueType = "zero";`<br>`}`<br>`else {  // no if here!!`<br>`  valueType = "positive";`<br>`}` |

## 4. The switch statement

If an if/else statement with multiple alternatives compares an int or char variable or expression against several constants you can use a switch statement.

**Syntax**

```
switch ( expression or value )
{
      case  value1:  set of statements;
                     ....
      case  value2:  set of statements;
                     ....
      case  value3:  set of statements;
                     ....
      case  value4:  set of statements;
                     ....
      case  value5:  set of statements;
                     ....
      .
      .
      default: set of statements;
}
```




```
switch (option) {
        case 'a':
                // statements if option='a'
                break;
        case 'b':
                // statements if option='b'
                break;
        case 'c':
                // statements if option='c'
                break;
        case 'd':
                // statements if option='d'
                break;
        default:
                // Else statement;
}
```

The break; statement will break the execution and exit the switch statement and continue with statement after the switch. If you do not provide break, execution will go to the satisfying condition and continue without break.

## Example: Calculator

```
#include <stdio.h>
int main(){
	int a, b;
	char opt;

	printf("Enter two numbers?");
	scanf("%d %d\n",&a,&b); // ‘\n’ may be required sometimes to flush buffer

	printf("enter the options (+ for addition, - for subtraction): ");
	scanf("%c",&opt);

	switch(opt){
		case '+':
			printf("Sum of %d and %d is %d", a,b, a+b);
			break;  // also try without break
		case '-':
			printf("Difference between %d and %d is %d", a,b, a-b);
			break;
		default:
			printf("Invalid option");
	}

}
```